Project 2 Report

Sarah Pierpoint and Melanie Vogt

October 13, 2026

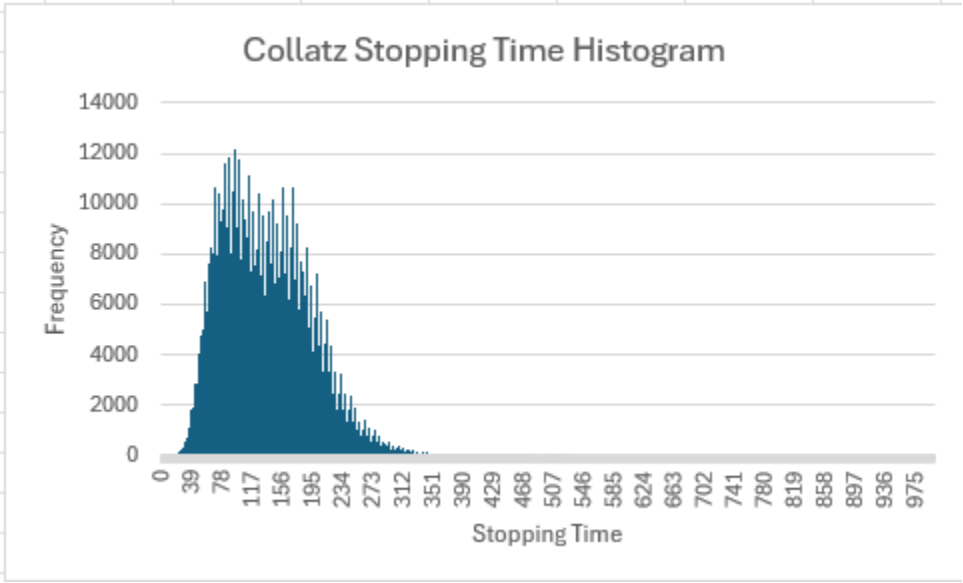
Experiment

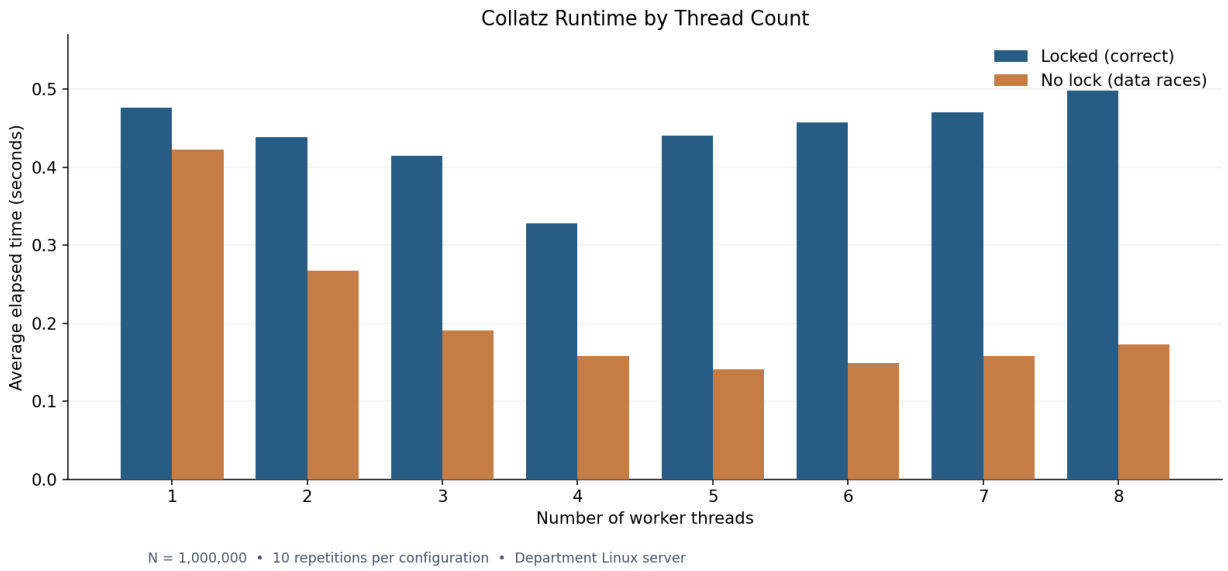
The experiments were run on the University of West Florida Computer Science Department's Linux server (cs-ssh.uwf.edu). It is a VMware virtual machine with 48 virtual CPUs (Intel Xeon Gold 6338N at 2.20 GHz) and 125 GiB of memory. Because the machine is shared with other users, other running programs can affect timing, which is why each configuration was run multiple times.

We tested the multithreaded Collatz stopping-time generator using N = 1,000,000 and thread counts ranging from 1 to 8. For each thread count we ran the program with mutex locking enabled and with the -nolock option, repeating each configuration 10 times for a total of 160 iterations. A bash script automated the experiments and recorded the elapsed time for each execution in a CSV file. A Bash script recorded the elapsed time for each execution in a CSV file. We calculated the average runtime for each configuration to examine how parallelism and synchronization affected performance.

Results

The histogram was computed for N = 1,000,000, and the longest stopping time was 524. The distribution is a hump peaking around 100-120 with a long right tail.





The runtime experiments showed that the fastest average execution with mutex locking occurred with 4 threads at approximately 0.3283 seconds, compared with 0.4767 seconds using 1 thread. Without locks the fastest average was approximately 0.1408 seconds with 5 threads. However, the unlocked version was not consistently correct. In a separate test using N=500,000, the unlocked 5 thread program recorded 670,184 histogram counts as opposed to the expected 500,000, but the locked version produced the correct result.

Discussion

The results reveal that multithreading can improve performance, however, increasing the number of threads does not always lead to faster execution. With the mutex locking enabled the runtime improved from 1 to 4 threads before increasing again with additional threads. This may be caused by contention for the shared counter and histogram, along with thread scheduling and synchronization overhead. The unlocked version usually ran faster due to the threads not needing to wait to acquire mutexes, but its results were unreliable due to race conditions. Protecting both shared resources was necessary to prevent multiple threads from claiming the same number or interfering with histogram updates. These results demonstrate the tradeoff between parallel performance and correct synchronization.

Conclusions

This experiment demonstrated how with multithreading you can distribute computational work across available processors while presenting challenges when threads access shared memory. Although the server exposed 48 logical CPUs, our program achieved its best synchronized performance with 4 worker threads. Adding threads did not continue to improve performance, showing that available CPU resources alone do not determine the efficiency of a parallel program.

Mutex locks were essential for producing reliable histogram results. Although disabling locks sometimes reduce execution time, it also introduced race conditions that caused incorrect counts. By protecting the shared counter and histogram while allowing Collatz calculations to run outside the critical sections, we

were able to maintain correctness and allow concurrent computation. Overall, the experiment showed that effective parallel programming requires balancing synchronization, correctness, and performance.